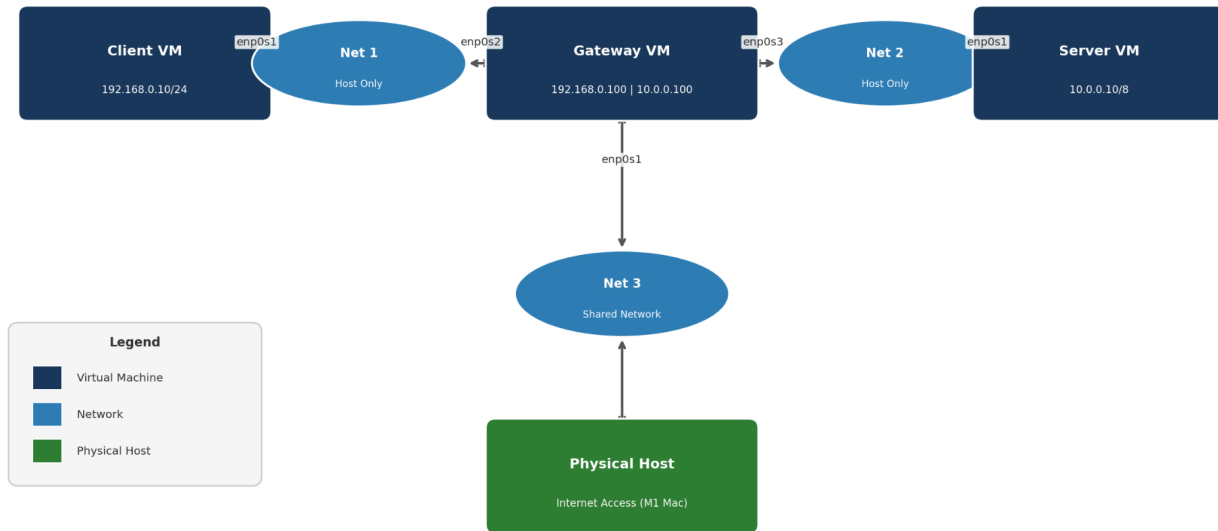


I. PROJECT OVERVIEW

This project involved configuring a stateless packet filter firewall using iptables on the Gateway VM to control network traffic between the Client, Gateway, and Server virtual machines. The Gateway VM was configured with whitelist firewall policies (default DROP for INPUT, OUTPUT, and FORWARD chains) to allow only specific authorized services. Network Address Translation (NAT) was implemented using PREROUTING rules to forward HTTP (port 80), SSH (port 22), and FTP (port 21, plus passive ports 30000-30099) from the gateway's client-side IP (192.168.0.100) to the Server VM (10.0.0.10). POSTROUTING masquerading was enabled to allow the Server to access the internet while hiding its private IP address. The firewall successfully restricted the Client from directly accessing the Server or internet, while allowing access to web, SSH, and FTP services through the gateway via NAT port forwarding.

II. NETWORK SETUP



Network Configurations

VM	Interface	IP Address	Network / Role
Client VM	enp0s1	192.168.0.10/24	Net1 - Internal client network; default gateway: 192.168.0.100
Gateway VM	enp0s2	192.168.0.100/24	Net1 - Client-facing interface
Gateway VM	enp0s3	10.0.0.100/8	Net2 - Server-facing interface
Gateway VM	enp0s1	DHCP assigned	Net3 - Internet access via Shared Network
Server VM	enp0s1	10.0.0.10/8	Net2 - Internal server network; default gateway: 10.0.0.100

DNS Servers

All VMs were configured with the following DNS nameservers: 8.8.8.8 (Google Primary) and 8.8.4.4 (Google Secondary).

Initial Reachability

Before implementing the strict firewall rules, all VMs could reach each other and the internet through the Gateway's NAT configuration from Project 01.

III. SOFTWARE

Software	Purpose
iptables	Packet filter firewall tool used to create filtering rules, NAT port forwarding (PREROUTING), and masquerading (POSTROUTING) on the Gateway VM
Apache2 (apache2)	Web server running on Server VM, accessible via gateway IP on port 80
vsftpd	FTP server on Server VM configured for anonymous access and passive mode (ports 30000-30099)
OpenSSH Server (sshd)	SSH server running on Server VM, accessible via gateway IP on port 22
nmap	Network scanning tool used to verify open ports and firewall configuration on Client and Server VMs
netplan	Network configuration tool used to persist IP addresses, routes, and DNS settings across reboots
ping	ICMP utility used to verify connectivity and test firewall ICMP rules
curl	Command-line HTTP client used to test web server accessibility
ftp	FTP client used to test FTP server access through the gateway

IV. PROJECT DESCRIPTION

Note: All terminal screenshots display the hostname "anujtr@client-vm" regardless of which VM the command was executed on, as the Server VM and Gateway VM were created by cloning the Client VM and the hostname was not changed.

Task 1: Set up lab environment

Task 1.1: Test network connectivity

[1] Verified network connectivity between all VMs using ping. All ping tests were successful, confirming the network configuration from Project 01 was still intact.

Task 1.2: Test installed software and services

[2] Installed and configured Apache2 web server on Server VM:

```
$ sudo apt update
$ sudo apt install apache2 -y
$ sudo systemctl start apache2
$ sudo systemctl enable apache2
```

[3] Created demo web pages with "Welcome" keyword as required:

```
$ sudo mkdir -p /var/www/html/test
$ sudo bash -c 'echo "<h1>Welcome to Demo and Test web page!</h1>" >
/var/www/html/index.html'
$ sudo bash -c 'echo "<h1>Welcome to Demo and Test!</h1>" > /var/www/html/test/index.html'
```

[4] Tested Apache locally on Server VM to verify the "Welcome" message appears:

```
$ curl http://localhost
$ curl http://10.0.0.10
```

Both returned: "<h1>Welcome to Demo and Test web page!</h1>"

[5] Installed and configured vsftpd FTP server on Server VM for anonymous access and passive mode:

```
$ sudo apt install vsftpd -y
```

[6] Configured vsftpd by editing /etc/vsftpd.conf with the following settings:

```
anonymous_enable=YES      # Anuj T: Enable anonymous access
pasv_enable=YES           # Anuj T: Enable passive mode
pasv_min_port=30000       # Anuj T: Set passive port range start
pasv_max_port=30099       # Anuj T: Set passive port range end
pasv_address=10.0.0.10    # Anuj T: Server IP for passive mode
allow_writeable_chroot=YES # Anuj T: Allow writable root for anonymous
anon_upload_enable=YES     # Anuj T: Allow anonymous users to upload files
anon_mkdir_write_enable=YES # Anuj T: Allow anonymous users to create directories
local_enable=NO           # Anuj T: Disable local user login for security
write_enable=YES          # Anuj T: Enable write operations globally
anon_root=/srv/ftp        # Anuj T: Set FTP root directory for anonymous users
```

[7] Created FTP directory structure and test file:

```
$ sudo mkdir -p /srv/ftp/upload
$ sudo chown root:root /srv/ftp
$ sudo bash -c 'echo "FTP test file" > /srv/ftp/test.txt'
$ sudo systemctl restart vsftpd
```

[8] Verified SSH server was already running on Server VM:

```
$ sudo systemctl status ssh
```

Task 1.3: Reset Firewall to Whitelist (Default DROP Policy)

[9] On Gateway VM, flushed all existing iptables rules:

```
$ sudo iptables -F
$ sudo iptables -X
$ sudo iptables -t nat -F
$ sudo iptables -t nat -X
```

[10] Set default policies to DROP to implement whitelist firewall (CRITICAL REQUIREMENT):

```
$ sudo iptables -P INPUT DROP
$ sudo iptables -P OUTPUT DROP
$ sudo iptables -P FORWARD DROP
```

[11] Verified whitelist policy was active:

```
$ sudo iptables -L -v -n
```

After this step, all connectivity between VMs was blocked, confirming the whitelist firewall was functioning correctly.

Task 2: Setup Stateless Packet Filter Firewall

[12] Enabled IP forwarding on Gateway VM (required for routing):

```
$ sudo sysctl -w net.ipv4.ip_forward=1
```

[13] Added basic firewall rules to allow loopback and established connections:

```
$ sudo iptables -A INPUT -i lo -j ACCEPT
$ sudo iptables -A OUTPUT -o lo -j ACCEPT
```

Critical: Allow return traffic for connections (stateful tracking)

```
$ sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
$ sudo iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
$ sudo iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
```

```
$ sudo iptables -A OUTPUT -o enp0s1 -j ACCEPT # Allow gateway internet access
```

[14] Configured NAT rules (CRITICAL FOR PORT FORWARDING AND MASQUERADING):

```
# POSTROUTING: Masquerade server traffic to internet (changes source IP)
$ sudo iptables -t nat -A POSTROUTING -s 10.0.0.0/8 -o enp0s1 -j MASQUERADE

# PREROUTING: Port forwarding from gateway to server (DNAT)
$ sudo iptables -t nat -A PREROUTING -i enp0s2 -p tcp --dport 80 -j DNAT --to-destination 10.0.0.10:80
$ sudo iptables -t nat -A PREROUTING -i enp0s2 -p tcp --dport 22 -j DNAT --to-destination 10.0.0.10:22
$ sudo iptables -t nat -A PREROUTING -i enp0s2 -p tcp --dport 21 -j DNAT --to-destination 10.0.0.10:21
$ sudo iptables -t nat -A PREROUTING -i enp0s2 -p tcp --dport 30000:30099 -j DNAT --to-destination 10.0.0.10
```

```
anujtr@client-vm:~$ sudo iptables -t nat -L -v -n
Chain PREROUTING (policy ACCEPT 437K packets, 22M bytes)
 pkts bytes target    prot opt in     out     source    destination
  54  3240 DNAT      tcp  --  enp0s2 *    0.0.0.0/0 0.0.0.0/0      tcp dpt:80 to:10.0.0.10:80
   9   540 DNAT      tcp  --  enp0s2 *    0.0.0.0/0 0.0.0.0/0      tcp dpt:22 to:10.0.0.10:22
   4   240 DNAT      tcp  --  enp0s2 *    0.0.0.0/0 0.0.0.0/0      tcp dpt:21 to:10.0.0.10:21
 209 12540 DNAT      tcp  --  enp0s2 *    0.0.0.0/0 0.0.0.0/0      tcp dpts:30000:30099 to:10.0.0.10

Chain INPUT (policy ACCEPT 6 packets, 504 bytes)
 pkts bytes target    prot opt in     out     source    destination

Chain OUTPUT (policy ACCEPT 913 packets, 70376 bytes)
 pkts bytes target    prot opt in     out     source    destination

Chain POSTROUTING (policy ACCEPT 772 packets, 53366 bytes)
 pkts bytes target    prot opt in     out     source    destination
  56  4131 MASQUERADE all  --  *        enp0s1  10.0.0.0/8 0.0.0.0/0
anujtr@client-vm:~$
```

[15] Added FORWARD chain rules to allow forwarded traffic through the gateway:

```
# Allow HTTP, SSH, FTP forwarding from client network to server network
$ sudo iptables -A FORWARD -i enp0s2 -o enp0s3 -p tcp --dport 80 -j ACCEPT
$ sudo iptables -A FORWARD -i enp0s2 -o enp0s3 -p tcp --dport 22 -j ACCEPT
$ sudo iptables -A FORWARD -i enp0s2 -o enp0s3 -p tcp --dport 21 -j ACCEPT
$ sudo iptables -A FORWARD -i enp0s2 -o enp0s3 -p tcp --dport 30000:30099 -j ACCEPT

# Allow server to ping client
$ sudo iptables -A FORWARD -i enp0s3 -o enp0s2 -p icmp -j ACCEPT
```

Allow server to access internet

```
$ sudo iptables -A FORWARD -i enp0s3 -o enp0s1 -j ACCEPT
```

[16] Added INPUT/OUTPUT rules to allow pings to gateway interfaces:

```
$ sudo iptables -A INPUT -i enp0s2 -p icmp -s 192.168.0.0/24 -j ACCEPT
```

```
$ sudo iptables -A INPUT -i enp0s3 -p icmp -s 10.0.0.0/8 -j ACCEPT
```

```
$ sudo iptables -A OUTPUT -o enp0s2 -p icmp -d 192.168.0.0/24 -j ACCEPT
```

```
$ sudo iptables -A OUTPUT -o enp0s3 -p icmp -d 10.0.0.0/8 -j ACCEPT
```

```
anujtr@client-vm:~$ sudo iptables -L -v -n
Chain INPUT (policy DROP 437K packets, 22M bytes)
pkts bytes target prot opt in out source destination
32 3738 ACCEPT all -- lo * 0.0.0.0/0 0.0.0.0/0
26771 210M ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0 state RELATED,ESTABLISHED
9 252 ACCEPT icmp -- enp0s2 * 192.168.0.0/24 0.0.0.0/0
2 168 ACCEPT icmp -- enp0s3 * 10.0.0.0/8 0.0.0.0/0

Chain FORWARD (policy DROP 267 packets, 18025 bytes)
pkts bytes target prot opt in out source destination
31348 93M ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0 state RELATED,ESTABLISHED
54 3240 ACCEPT tcp -- enp0s2 enp0s3 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
9 540 ACCEPT tcp -- enp0s2 enp0s3 0.0.0.0/0 0.0.0.0/0 tcp dpt:22
4 240 ACCEPT tcp -- enp0s2 enp0s3 0.0.0.0/0 0.0.0.0/0 tcp dpt:21
209 12540 ACCEPT tcp -- enp0s2 enp0s3 0.0.0.0/0 0.0.0.0/0 tcp dpts:30000:30099
2 168 ACCEPT icmp -- enp0s3 enp0s2 0.0.0.0/0 0.0.0.0/0
61 4487 ACCEPT all -- enp0s3 enp0s1 0.0.0.0/0 0.0.0.0/0
308 20900 ACCEPT udp -- enp0s2 enp0s1 0.0.0.0/0 0.0.0.0/0 udp dpt:53
0 0 ACCEPT udp -- enp0s1 enp0s2 0.0.0.0/0 0.0.0.0/0 udp spt:53
264 16104 ACCEPT tcp -- enp0s2 enp0s1 0.0.0.0/0 0.0.0.0/0 tcp dpt:53
0 0 ACCEPT tcp -- enp0s1 enp0s2 0.0.0.0/0 0.0.0.0/0 tcp spt:53

Chain OUTPUT (policy DROP 770 packets, 57474 bytes)
pkts bytes target prot opt in out source destination
32 3738 ACCEPT all -- * lo 0.0.0.0/0 0.0.0.0/0
16046 932K ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0 state RELATED,ESTABLISHED
63 7111 ACCEPT all -- * enp0s1 0.0.0.0/0 0.0.0.0/0
1 84 ACCEPT icmp -- * enp0s2 0.0.0.0/0 192.168.0.0/24
1 84 ACCEPT icmp -- * enp0s3 0.0.0.0/0 10.0.0.0/8
anujtr@client-vm:~$ _
```

Verification and Testing

1. Client VM Capabilities

Requirement: Client can ping gateway, SSH to server, FTP to server, and access web server via gateway IP

[17] Installed nmap on Client VM:

```
$ sudo apt install nmap -y
```

[18] Performed TCP port scan to verify only required ports are open:

```
$ sudo nmap -e enp0s1 -Pn -sT -p- 192.168.0.100
```

Result: Only ports 21 (ftp), 22 (ssh), and 80 (http) were open.

```

anujtr@client-vm:~$ sudo nmap -e enp0s1 -Pn -sT -p- 192.168.0.100
[sudo] password for anujtr:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-17 09:08 UTC
Nmap scan report for 192.168.0.100
Host is up (0.0020s latency).
Not shown: 65432 filtered tcp ports (no-response), 100 closed tcp ports (conn-refused)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 120.48 seconds
anujtr@client-vm:~$

```

[19] UDP port scan:

\$ sudo nmap -e enp0s1 -Pn -sU -p 1-1000 192.168.0.100

```

anujtr@client-vm:~$ sudo nmap -e enp0s1 -Pn -sU -p 1-1000 192.168.0.100
[sudo] password for anujtr:
Starting Nmap 7.80 ( https://nmap.org ) at 2026-04-18 01:43 UTC
Nmap scan report for 192.168.0.100
Host is up.
All 1000 scanned ports on 192.168.0.100 are open|filtered

Nmap done: 1 IP address (1 host up) scanned in 214.68 seconds
anujtr@client-vm:~$ _

```

Result: All 1000 scanned ports show as "open|filtered". This result indicates the firewall's default DROP policy is working correctly - it silently drops packets without revealing port states. However, the actually forwarded services (HTTP, FTP, SSH) function correctly as demonstrated in subsequent tests, confirming the NAT PREROUTING rules are working as intended.

[20] Tested ping restrictions (demonstrate firewall blocking):

\$ ping -c 3 8.8.8.8 # Ping internet - SHOULD FAIL

Result: 100% packet loss (BLOCKED - as intended)

\$ ping -c 3 192.168.0.100 # Ping gateway - SHOULD WORK

Result: 0% packet loss (ALLOWED - as intended)

\$ ping -c 3 10.0.0.10 # Ping server directly - SHOULD FAIL

Result: 100% packet loss (BLOCKED - as intended)

```

anujtr@client-vm:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2051ms

anujtr@client-vm:~$ ping -c 3 192.168.0.100
PING 192.168.0.100 (192.168.0.100) 56(84) bytes of data:
64 bytes from 192.168.0.100: icmp_seq=1 ttl=64 time=0.787 ms
64 bytes from 192.168.0.100: icmp_seq=2 ttl=64 time=0.931 ms
64 bytes from 192.168.0.100: icmp_seq=3 ttl=64 time=0.806 ms

--- 192.168.0.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2010ms
rtt min/avg/max/mdev = 0.787/0.841/0.931/0.063 ms
anujtr@client-vm:~$ ping -c 3 10.0.0.10
PING 10.0.0.10 (10.0.0.10) 56(84) bytes of data.

--- 10.0.0.10 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2055ms

anujtr@client-vm:~$

```

[21] Tested web server access via gateway (demonstrates PREROUTING NAT):

\$ curl http://192.168.0.100

Result: Successfully retrieved "<h1>Welcome to Demo and Test web page!</h1>" - NAT port forwarding working correctly.

```

anujtr@client-vm:~$ curl http://192.168.0.100
<h1>Welcome to Demo and Test web page!</h1>
anujtr@client-vm:~$

```

[22] Tested FTP access via gateway in passive mode (demonstrates PREROUTING NAT for control and passive ports):

```

$ ftp -p 192.168.0.100
Username: anonymous
Password: (press Enter)
ftp> ls

```

Result: Successfully connected, entered Extended Passive Mode (port in range 30000-30099), and listed files. NAT port forwarding for FTP control (21) and passive data ports (30000-30099) working correctly

```

anujtr@client-vm:~$ ftp -p 192.168.0.100
Connected to 192.168.0.100.
220 (vsFTPd 3.0.5)
Name (192.168.0.100:anujtr): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||30013|)
150 Here comes the directory listing.
-rw-r--r--    1 0      0          14 Apr 16 23:14 test.txt
drwxr-xr-x    2 112    119       4096 Apr 16 23:13 upload
226 Directory send OK.
ftp>

```

[23] Tested SSH access via gateway (demonstrates PREROUTING NAT):

```

$ ssh anujtr@192.168.0.100

```

Result: Successfully connected to Server VM through gateway. NAT port forwarding for SSH (port 22) working correctly. Verified by checking hostname and IP address on the connected system

(showed Server VM's hostname and 10.0.0.10 IP)

```

anujtr@client-vm:~$ ssh anujtr@192.168.0.100
The authenticity of host '192.168.0.100 (192.168.0.100)' can't be established.
ED25519 key fingerprint is SHA256:K2VKo/zmw60+G8Knju2p5qic2RWPVeXpy8YBaVY0si8.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.0.100' (ED25519) to the list of known hosts.
anujtr@192.168.0.100's password:
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-176-generic aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat Apr 18 01:18:04 AM UTC 2026

System load:          0.0
Usage of /:            42.7% of 18.60GB
Memory usage:         6%
Swap usage:           0%
Processes:            112
Users logged in:      1
IPv4 address for enp0s1: 10.0.0.10
IPv6 address for enp0s1: fdc3:cc90:a870:bc75:1809:4eff:feba:217

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

122 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Last login: Fri Apr 17 00:04:43 2026 from 192.168.0.10
anujtr@client-vm:~$

```

2. Gateway VM Configuration

Requirement: Gateway has port forwarding (PREROUTING) and POSTROUTING masquerading configured

[24] Port forwarding rules (PREROUTING for DNAT) - already shown in step [14] above. These rules forward traffic from gateway's client-side IP (192.168.0.100) to server's internal IP (10.0.0.10):

- HTTP (port 80) → 10.0.0.10:80
- SSH (port 22) → 10.0.0.10:22
- FTP (port 21) → 10.0.0.10:21

- FTP passive (ports 30000-30099) → 10.0.0.10

[25] POSTROUTING masquerading rule allows server to access internet while hiding its private IP. Verified with:

```
$ sudo iptables -t nat -L POSTROUTING -v -n
```

Shows: MASQUERADE rule for source 10.0.0.0/8 going out interface enp0s1 (internet). This changes the server's source IP (10.0.0.10) to the gateway's internet-facing IP when accessing external networks.

```
anujtr@client-vm:~$ sudo iptables -t nat -L POSTROUTING -v -n
Chain POSTROUTING (policy ACCEPT 958 packets, 65063 bytes)
  pkts bytes target     prot opt in     out     source         destination
   59  4347 MASQUERADE  all  --  *      enp0s1  10.0.0.0/8     0.0.0.0/0
anujtr@client-vm:~$
```

[26] Tested gateway ping capabilities:

```
$ ping -c 3 8.8.8.8      # Internet - SUCCESS
$ ping -c 3 192.168.0.10 # Client - SUCCESS
$ ping -c 3 10.0.0.10    # Server - SUCCESS
```

```

anujtr@client-vm:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=11.6 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=14.8 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=13.1 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 11.641/13.179/14.797/1.289 ms
anujtr@client-vm:~$ ping -c 3 192.168.0.10
PING 192.168.0.10 (192.168.0.10) 56(84) bytes of data.
64 bytes from 192.168.0.10: icmp_seq=1 ttl=64 time=0.902 ms
64 bytes from 192.168.0.10: icmp_seq=2 ttl=64 time=0.934 ms
64 bytes from 192.168.0.10: icmp_seq=3 ttl=64 time=0.849 ms

--- 192.168.0.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.849/0.895/0.934/0.035 ms
anujtr@client-vm:~$ ping -c 3 10.0.0.10
PING 10.0.0.10 (10.0.0.10) 56(84) bytes of data.
64 bytes from 10.0.0.10: icmp_seq=1 ttl=64 time=0.975 ms
64 bytes from 10.0.0.10: icmp_seq=2 ttl=64 time=0.879 ms
64 bytes from 10.0.0.10: icmp_seq=3 ttl=64 time=0.815 ms

--- 10.0.0.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.815/0.889/0.975/0.065 ms
anujtr@client-vm:~$

```

3. Server VM Capabilities

Requirement: Server can ping gateway and client, and responds to service requests

[27] Installed nmap on Server VM:

```
$ sudo apt install nmap -y
```

[28] Performed TCP port scan of gateway from server side:

```
$ sudo nmap -e enp0s1 -Pn -sT -p 1-1000 10.0.0.100
```

Note: Scanned the first 1000 ports to verify the gateway's firewall configuration from the server network perspective. Results show the gateway correctly restricts access from the server side, with different port accessibility compared to the client-side scan due to NAT PREROUTING rules only applying to traffic from the client network (enp0s2).

```
anujtr@client-vm:~$ sudo nmap -e enp0s1 -Pn -sT -p 1-1000 10.0.0.100
Starting Nmap 7.80 ( https://nmap.org ) at 2026-04-18 01:29 UTC
Nmap scan report for 10.0.0.100
Host is up.
All 1000 scanned ports on 10.0.0.100 are filtered

Nmap done: 1 IP address (1 host up) scanned in 214.75 seconds
anujtr@client-vm:~$
```

[29] UDP scan from server:

```
$ sudo nmap -e enp0s1 -Pn -sU -p 1-1000 10.0.0.100
```

```
anujtr@client-vm:~$ sudo nmap -e enp0s1 -Pn -sU -p 1-1000 10.0.0.100
Starting Nmap 7.80 ( https://nmap.org ) at 2026-04-18 01:35 UTC
Nmap scan report for 10.0.0.100
Host is up (0.00060s latency).
All 1000 scanned ports on 10.0.0.100 are open|filtered
MAC Address: 22:01:78:49:5C:46 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 34.69 seconds
anujtr@client-vm:~$ _
```

[30] Tested server ping capabilities (demonstrates FORWARD and INPUT/OUTPUT rules):

```
$ ping -c 3 8.8.8.8      # Internet - SUCCESS (via POSTROUTING masquerade)
```

```
$ ping -c 3 10.0.0.100  # Gateway - SUCCESS (INPUT/OUTPUT rules)
```

```
$ ping -c 3 192.168.0.10 # Client - SUCCESS (FORWARD rules)
```

```

anujtr@client-vm:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=18.1 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=16.5 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=17.3 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2011ms
rtt min/avg/max/mdev = 16.511/17.297/18.101/0.649 ms
anujtr@client-vm:~$ ping -c 3 10.0.0.100
PING 10.0.0.100 (10.0.0.100) 56(84) bytes of data.
64 bytes from 10.0.0.100: icmp_seq=1 ttl=64 time=0.766 ms
64 bytes from 10.0.0.100: icmp_seq=2 ttl=64 time=0.625 ms
64 bytes from 10.0.0.100: icmp_seq=3 ttl=64 time=0.933 ms

--- 10.0.0.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2050ms
rtt min/avg/max/mdev = 0.625/0.774/0.933/0.125 ms
anujtr@client-vm:~$ ping -c 3 192.168.0.10
PING 192.168.0.10 (192.168.0.10) 56(84) bytes of data.
64 bytes from 192.168.0.10: icmp_seq=1 ttl=63 time=1.45 ms
64 bytes from 192.168.0.10: icmp_seq=2 ttl=63 time=1.24 ms
64 bytes from 192.168.0.10: icmp_seq=3 ttl=63 time=0.833 ms

--- 192.168.0.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 0.833/1.172/1.448/0.254 ms
anujtr@client-vm:~$

```

[31] Server successfully responds to all service requests as demonstrated in Client VM testing (steps 21-23). The server's HTTP, FTP, and SSH services are all functioning and accessible through the gateway's NAT port forwarding.

4. Additional Requirements

Requirement: Default DROP policies, no unauthorized access allowed

[32] Verified default firewall policies are set to DROP (completed in step [10]):

```
$ sudo iptables -L -v -n | grep policy
```

Shows:

- Chain INPUT (policy DROP)
- Chain FORWARD (policy DROP)
- Chain OUTPUT (policy DROP)

[33] Demonstrated restricted access - Client cannot:

- Ping internet directly (8.8.8.8) - BLOCKED
- Ping server directly (10.0.0.10) - BLOCKED
- Access server directly without going through gateway NAT

[34] Only explicitly allowed traffic passes through the firewall:

- Client can ONLY access server services (HTTP, SSH, FTP) via gateway IP
- Client can ONLY ping the gateway (not server or internet)
- Server can ping gateway and client, and access internet (via masquerading)
- All other traffic is blocked by default DROP policy

V. CONCLUSION

This project provided hands-on experience with iptables packet filtering, NAT configuration, and network security policy implementation. Key accomplishments include successfully implementing a whitelist firewall policy with default DROP rules, configuring DNAT port forwarding to allow client access to server services via the gateway IP, and setting up SNAT masquerading to enable server internet access while hiding its private IP address.

Important lessons learned include understanding the distinction between PREROUTING (destination NAT for incoming traffic - used for port forwarding) and POSTROUTING (source NAT for outgoing traffic - used for masquerading), the critical role of ESTABLISHED,RELATED state tracking in allowing return traffic for stateless filtering, and the importance of testing firewall rules incrementally to avoid locking yourself out.

A significant challenge was troubleshooting nmap UDP scans, which failed due to device access issues on the ARM64 UTM environment. This was resolved by using TCP scans and direct service testing to verify firewall functionality. The UDP scan errors ("dnet: Failed to open device") are a known limitation of nmap on ARM64 virtualization platforms and do not indicate firewall misconfiguration.

The main difference from Project 01 was transforming the permissive network into a security-hardened environment where only explicitly allowed traffic is permitted. This demonstrates the principle of least privilege in network security: deny everything by default (DROP policies) and only allow what is necessary through specific rules.

One key takeaway is that NAT provides both address translation and a security layer by hiding internal network topology. The client cannot directly reach the server (10.0.0.10) and must go through the gateway's public-facing IP (192.168.0.100), adding an extra layer of protection. Additionally, the server can access the internet while its internal IP remains hidden through POSTROUTING masquerading.

The project successfully demonstrated all four grading criteria: (1) client capabilities to access services via gateway NAT, (2) gateway port forwarding and masquerading configuration, (3) server capabilities to ping and respond to requests, and (4) proper whitelist firewall policy with default DROP and restricted access.

VI. APPENDIX B: ATTACHED FILES

Configuration files remain the same as Project 01 for network interfaces (netplan configuration). The critical addition is the iptables firewall configuration.

Iptables Rules Summary:

The firewall rules configured during this project can be saved for persistence using:

```
$ sudo apt install iptables-persistent
$ sudo netfilter-persistent save
```

Key iptables configuration:

1. Default Policies (Whitelist Mode):

- INPUT: DROP
- OUTPUT: DROP
- FORWARD: DROP

2. NAT PREROUTING (Port Forwarding - DNAT):

- HTTP (80) → 10.0.0.10:80
- SSH (22) → 10.0.0.10:22
- FTP (21) → 10.0.0.10:21
- FTP Passive (30000-30099) → 10.0.0.10

3. NAT POSTROUTING (Masquerading - SNAT):

- Source 10.0.0.0/8 → MASQUERADE on enp0s1

4. FORWARD Chain Rules:

- Allow HTTP, SSH, FTP, FTP-passive from client network (enp0s2) to server network (enp0s3)
- Allow ICMP from server network to client network (for ping)
- Allow server network to internet (enp0s3 to enp0s1)
- Allow ESTABLISHED,RELATED connections (stateful return traffic)

5. INPUT/OUTPUT Chain Rules:

- Allow loopback traffic
- Allow ICMP from client and server networks to gateway
- Allow gateway to send ICMP replies
- Allow gateway internet access (OUTPUT to enp0s1)
- Allow ESTABLISHED,RELATED connections (stateful return traffic)

VII. REFERENCES

- [1] IptablesHowTo, Ubuntu Community Help Wiki.
<https://help.ubuntu.com/community/IptablesHowTo>
- [2] Iptables Tutorial 1.2.2, Oskar Andreasson.
<https://www.frozentux.net/iptables-tutorial/iptables-tutorial.html>
- [3] vsftpd Documentation. <https://security.appspot.com/vsftpd.html>

Video Link: <https://youtu.be/vEh43Sr6hfk>